BookingApp

Documentacion Tecnica

Plataforma SaaS de Reservas Multi-Tenant

Autor	**Anass El Morabit**
GitHub	**github.com/ANASSELMORABIT**
Version	**1.0.0**
Fecha	**2026**
Estado	**Production Ready**

ASP.NET Core 10 | React + TypeScript | PostgreSQL | Docker | SignalR

1. Descripcion General

BookingApp es una plataforma SaaS (Software as a Service) de gestion de reservas multi-tenant diseñada para negocios de servicios como peluquerias, gimnasios, clinicas y espacios de coworking. La plataforma permite a multiples negocios gestionar sus reservas, personal, servicios y estadisticas desde un unico sistema centralizado.

Caracteristicas principales:

- Arquitectura multi-tenant: cada negocio tiene su propio subdominio y datos aislados
- Sistema de roles granular: SuperAdmin, Owner, Staff y Client con permisos diferenciados
- Calendario visual interactivo con vista diaria, semanal y mensual
- Pagina publica de reservas sin necesidad de cuenta de usuario
- Notificaciones en tiempo real via WebSockets (SignalR)
- Exportacion de datos a Excel y PDF con estadisticas detalladas
- Dashboard con KPIs, graficas y analisis de rendimiento
- Sistema de pagos integrado con Stripe (modo test)
- Email reminders automaticos con Hangfire (jobs en background)

2. Arquitectura del Sistema

Clean Architecture — 4 Capas

El proyecto sigue los principios de Clean Architecture, separando las responsabilidades en capas independientes con dependencias unidireccionales.

Capa	Proyecto	Responsabilidad
Domain	BookingApp.Domain	Entidades, Enums, Contratos base
Application	BookingApp.Application	Interfaces, DTOs, Casos de uso
Infrastructure	BookingApp.Infrastructure	EF Core, Servicios, SignalR, Email
API	BookingApp.API	Controllers, Middleware, Hubs, Program.cs

Modelo de datos — Entidades principales

Entidad	Descripcion	Relaciones
Tenant	Negocio suscrito al SaaS	Has many Business, Users
Business	Local fisico del negocio	Belongs to Tenant, Has many Service, Staff
Service	Servicio ofrecido (corte, clase...)	Belongs to Business, Has many Booking

Staff	Empleado del negocio	Belongs to Business, Has many Booking
Booking	Reserva concreta	Belongs to Service, Staff, AppUser
AppUser	Usuario del sistema	Belongs to Tenant, Has many Booking

3. Stack Tecnologico

Backend

Tecnologia	Version	Uso
ASP.NET Core	10.0	Framework principal de la API REST
Entity Framework Core	10.0	ORM para acceso a base de datos
PostgreSQL	16	Base de datos relacional principal
SignalR	Built-in	Notificaciones en tiempo real (WebSockets)
Hangfire	1.8+	Jobs en background (email reminders)
JWT Bearer	10.0	Autenticacion y autorizacion
BCrypt.Net	4.0	Hash seguro de contrasenas
Stripe.net	Latest	Integracion de pagos
ClosedXML	Latest	Exportacion a Excel (.xlsx)
QuestPDF	Latest	Generacion de informes PDF
Docker	24+	Contenedorizacion y despliegue

Frontend

Tecnologia	Version	Uso
React	18	Framework UI principal
TypeScript	5+	Tipado estatico
Vite	6+	Build tool y dev server
Tailwind CSS	4	Framework de estilos utility-first
Shadcn/ui	Latest	Componentes UI accesibles
TanStack Query	5	Gestion de estado del servidor
React Router	6	Enrutamiento SPA
Recharts	Latest	Graficas y visualizaciones
FullCalendar	Latest	Calendario visual interactivo
Microsoft SignalR	Latest	Cliente WebSocket para notificaciones

Axios	Latest	Cliente HTTP

4. Funcionalidades Detalladas

4.1 Sistema de Autenticacion y Roles

La aplicacion implementa autenticacion JWT con refresh tokens y un sistema de roles basado en politicas de autorizacion (Policy-based Authorization).

Rol	Permisos	Acceso
SuperAdmin	Acceso total al sistema	Todos los negocios y tenants
Owner	Gestion completa de su negocio	Sus negocios, staff y servicios
Staff	Ver y gestionar reservas y servicios	Sus reservas y servicios asignados
Client	Crear y cancelar reservas	Solo sus propias reservas

4.2 Sistema de Reservas

El nucleo del sistema es el motor de reservas que gestiona disponibilidad, conflictos y slots de tiempo respetando la zona horaria del negocio (Europe/Madrid).

- Algoritmo de slots: genera intervalos de tiempo basados en la duracion del servicio
- Control de conflictos: verifica solapamientos antes de confirmar una reserva
- Gestion de zonas horarias: convierte UTC a hora local para mostrar al usuario
- Estados: Pending, Confirmed, Cancelled, Completed con transiciones controladas
- Pagina publica: cualquier cliente puede reservar sin crear cuenta (/book/:subdomain)
- Flow en 4 pasos: seleccion de servicio, fecha/hora, datos personales y confirmacion

4.3 Panel de Administracion

Dashboard completo con KPIs en tiempo real y graficas interactivas para monitorizar el rendimiento del negocio.

- KPIs: reservas hoy, este mes, ingresos, tasa de cancelacion
- Graficas de barras: reservas confirmadas vs canceladas por dia
- Grafica de linea: evolucion de ingresos por dia
- Grafica de tarta: distribucion de estados de reservas
- Ranking de servicios: por numero de reservas e ingresos generados
- Rendimiento del staff: reservas completadas, canceladas e ingresos por empleado
- Filtros: por negocio y por periodo (7 dias, 30 dias, 3 meses, todo)
- Exportacion: Excel con todas las reservas y PDF con informe de estadisticas

4.4 Notificaciones en Tiempo Real

Implementadas con SignalR (WebSockets), las notificaciones se envian instantaneamente a todos los usuarios conectados del mismo tenant.

- Nueva reserva: notifica al Owner y SuperAdmin cuando un cliente reserva
- Reserva cancelada: alerta inmediata al Owner cuando se cancela una cita
- Reserva confirmada: confirmacion al cliente cuando el Owner confirma
- SuperAdmin recibe notificaciones de todos los tenants simultaneamente
- Campana con contador de no leidas en el sidebar de navegacion
- Panel desplegable con historial de las ultimas 20 notificaciones

5. API REST — Endpoints Principales

Metodo	Endpoint	Auth	Descripcion
POST	/api/Auth/register	Publico	Registro de nuevo usuario/tenant
POST	/api/Auth/login	Publico	Login y obtencion de JWT
GET	/api/Profile	JWT	Ver perfil del usuario conectado
PUT	/api/Profile	JWT	Actualizar nombre y telefono
PUT	/api/Profile/change-password	JWT	Cambiar contrasena
GET	/api/Business	JWT	Listar negocios del tenant
POST	/api/Business	Owner	Crear nuevo negocio
PUT	/api/Business/{id}	Owner	Editar negocio
GET	/api/Business/{id}/staff	Owner/Staff	Listar staff del negocio
POST	/api/Business/{id}/staff	Owner	Añadir nuevo empleado
GET	/api/Business/{id}/services	JWT	Listar servicios activos
POST	/api/Business/{id}/services	Owner/Staff	Crear servicio
GET	/api/Availability	Publico	Consultar slots disponibles
POST	/api/Bookings	JWT	Crear reserva (usuario autenticado)
GET	/api/Bookings/my	JWT	Ver mis reservas
DELETE	/api/Bookings/{id}	JWT	Cancelar reserva
GET	/api/Admin/dashboard	Owner/Staff	Estadisticas del negocio
GET	/api/Admin/bookings-by-period	Owner	Reservas por rango de fechas
GET	/api/Export/excel	Owner	Exportar reservas a Excel
GET	/api/Export/pdf	Owner	Exportar estadisticas a PDF
GET	/api/public/business/{subdomain}	Publico	Info publica del negocio
POST	/api/public/book	Publico	Reserva sin cuenta de usuario
WS	/hubs/notifications	JWT	Hub de notificaciones SignalR

6. Instalacion y Ejecucion Local

Requisitos previos

- .NET 10 SDK
- Node.js 20+
- Docker Desktop
- Git

Pasos de instalacion

1. Clonar el repositorio

```
git clone https://github.com/ANASSELMORABIT/BookingApp.git
cd BookingApp
```

2. Levantar la base de datos

```
docker-compose up -d
```

3. Aplicar migraciones

```
dotnet ef database update --project src\BookingApp.Infrastructure --startup-project
src\BookingApp.API
```

4. Arrancar el backend

```
dotnet run --project src\BookingApp.API
```

5. Arrancar el frontend

```
cd src\Booking.Frontend
npm install
npm run dev
```

6. Abrir en el navegador

```
http://localhost:5173
```

Variables de entorno (.env / appsettings.json)

Variable	Descripcion	Ejemplo
ConnectionStrings:DefaultConnection	Cadena de conexion PostgreSQL	Host=localhost;Port=5432;...
Jwt:Key	Clave secreta para firmar JWT	clave-minimo-32-caracteres
Jwt:Issuer	Emisor del token JWT	BookingApp
Jwt:Audience	Audiencia del token JWT	BookingAppUsers
Stripe:SecretKey	Clave secreta de Stripe	sk_test_...
Stripe:WebhookSecret	Secreto del webhook de Stripe	whsec_...

7. Estructura del Proyecto

```
BookingApp/
■■■ src/
■   ■■■ BookingApp.Domain/          # Entidades y Enums
■   ■   ■■■ Entities/               # Tenant, Business, Service, Staff, Booking,
AppUser
■   ■   ■■■ Enums/                  # BookingStatus, PaymentStatus
■   ■   ■■■ Common/                 # BaseEntity
■   ■■■ BookingApp.Application/     # Casos de uso e Interfaces
■   ■   ■■■ DTOs/                   # Auth, Bookings, Business, Admin, Profile...
■   ■   ■■■ Interfaces/             # IAuthService, IBookingService, IAdminServic
e...
■   ■■■ BookingApp.Infrastructure/  # Implementaciones
■   ■   ■■■ Persistence/            # AppDbContext, Configurations, Migrations
■   ■   ■■■ Services/               # AuthService, BookingService, ExportService.
..
■   ■   ■■■ Jobs/                   # BookingReminderJob (Hangfire)
■   ■■■ BookingApp.API/             # API REST
■   ■   ■■■ Controllers/            # Auth, Bookings, Business, Admin, Export, Pu
blic
■   ■   ■■■ Hubs/                   # NotificationHub (SignalR)
■   ■   ■■■ Program.cs
■   ■■■ Booking.Frontend/           # React + TypeScript
■       ■■■ src/
■           ■■■ api/                # client.ts, auth.ts, business.ts, admin.ts...
■           ■■■ components/         # layout/, ui/ (Shadcn)
■           ■■■ context/            # AuthContext.tsx
■           ■■■ hooks/              # useNotifications.ts
■           ■■■ pages/              # admin/, auth/, booking/, profile/, public/
■           ■■■ types/              # index.ts
■           ■■■ lib/                # utils.ts, businessIcons.tsx
■■■ tests/
■   ■■■ BookingApp.Tests/           # xUnit tests
■■■ docker-compose.yml
■■■ Dockerfile
■■■ BookingApp.slnx
```

8. Docker y Despliegue

La aplicacion esta completamente contenedorizada con Docker:

- Dockerfile multi-stage: build con SDK de .NET y runtime minimo en produccion
- docker-compose.yml: orquesta la API y PostgreSQL con health checks
- Variables de entorno: configuracion externalizada para diferentes entornos
- GitHub Actions: CI/CD automatico en cada push a main (build + test + docker build)

Comandos Docker utiles

Levantar todo en produccion:

```
docker-compose up --build -d
```

Ver logs de la API:

```
docker logs bookingapp_api
```

Conectarse a PostgreSQL:

```
docker exec -it bookingapp_db psql -U booking_user -d bookingapp
```

Parar todos los servicios:

```
docker-compose down
```

9. Testing

El proyecto incluye tests unitarios con xUnit que cubren la logica de negocio critica:

- Booking_OverlapsWith_ShouldReturnTrue_WhenOverlapping: valida deteccion de solapamientos
- Booking_OverlapsWith_ShouldReturnFalse_WhenNotOverlapping: valida slots libres correctamente
- Booking_Cancel_ShouldChangeStatus_ToCancelled: valida la transicion de estado al cancelar

```
dotnet test
```